Highlights

Homotopy-Based Functional Neural Networks for Robust Identification of Uncertain Nonlinear Dynamic Systems

Rodolfo H. Rodrigo, Gustavo Schweickardt, Daniel H. Patiño

- A homotopy-based algebraic regressor converts implicit nonlinear discrete equations into explicit trainable form.
- Gaussian RBF with analytical derivatives (N, N', N'') is embedded within the homotopy regressor.
- Training on 30 samples achieves MSE $< 10^{-8}$ on the Duffing oscillator benchmark.
- The HFNN requires 10–33$\times$ fewer samples than PINNs, Neural ODEs, and Narendra's model.
- Stability, zero-stability, and consistency are formally established for the proposed scheme.

Homotopy-Based Functional Neural Networks for Robust Identification of Uncertain Nonlinear Dynamic Systems

Rodolfo H. Rodrigo[a,*], Gustavo Schweickardt[b], Daniel H. Patiño[c]

[a] *Departamento Electromecánica, Facultad de Ingeniería, Universidad Nacional de San Juan, San Juan, Argentina*
[b] *CONICET / Universidad Tecnológica Nacional, Entre Ríos, Argentina*
[c] *Instituto de Automática (INAUT), Facultad de Ingeniería, Universidad Nacional de San Juan, San Juan, Argentina*

Abstract

This paper introduces a homotopy-based functional neural architecture, termed the Homotopy-Based Functional Neural Network (HFNN), for robust identification of nonlinear dynamic systems with partially known structure and bounded disturbances. Unlike conventional neural networks trained via backpropagation or adjoint methods, the HFNN leverages the Homotopy Analysis Method (HAM) to convert the implicit nonlinear discrete equation arising from a Linear Multistep Method (LMM) into an explicit algebraic regressor. The resulting rational-polynomial expression retains the nonlinear terms $N(y)$, $N'(y)$, and $N''(y)$ by construction, providing a theoretically founded discretization of nonlinear ODEs—analogous to what Tustin and zero-order hold provide for linear systems.

The trainable parameters of the HFNN—including Padé rational coefficients and the weights of an embedded Radial Basis Function (RBF) approximator—are optimized via Levenberg–Marquardt with analytically computed network derivatives. A formal parameter adaptation law, cost function, and training algorithm are presented. The convergence-control parameter $\hbar$ provides explicit regulation of the approximation domain.

*Corresponding author.

Email addresses: `rodolfo_rh@unsj.edu.ar` (Rodolfo H. Rodrigo), `gustavoschweickardt@yahoo.com.ar` (Gustavo Schweickardt), `dpatino@inaut.unsj.edu.ar` (Daniel H. Patiño)

The approach is validated on four case studies of increasing complexity: (i) a nonlinear first-order system, (ii) a nonlinear oscillator with sine-squared dynamics, (iii) an RBF-based approximation with parametric uncertainty, and (iv) a Duffing oscillator with an embedded Gaussian RBF network ($M = 10$ centres), where training with only 30 samples achieves MSE $< 10^{-8}$. Comparisons with Physics-Informed Neural Networks, Neural ODEs, and Narendra's series-parallel model demonstrate the data efficiency of the approach. The HFNN delivers accurate predictions, strong extrapolation, robustness to noise and parameter variations, and interpretability, positioning it as a data-efficient and theoretically grounded tool for grey-box system identification.

Keywords: Homotopy Analysis Method, Functional Neural Networks, System Identification, Nonlinear Dynamic Systems, Grey-Box Modeling, Algebraic Regressor, Radial Basis Functions

---

1. Introduction

The identification of nonlinear dynamic systems from measured data is a central problem in control engineering, signal processing, and applied mathematics. When the governing ordinary differential equations (ODEs) are fully known, classical numerical integration methods—Euler, Runge–Kutta, Adams–Bashforth—produce accurate trajectories with well-understood error bounds [1, 2]. However, when the system contains unknown nonlinear terms, the problem shifts from *simulation* (computing trajectories from known equations) to *identification* (determining model parameters from input–output data) [3].

For *linear* systems, this problem is well solved. Standard discretization methods (Euler, Tustin bilinear transform, zero-order hold) convert a continuous-time transfer function into an exact discrete-time equivalent—a linear difference equation whose coefficients correspond directly to the physical parameters. Training any parametric model inside such a regressor is straightforward and routinely done in adaptive and self-tuning control [4, 3].

For *nonlinear* systems, no comparable procedure existed. Consider the ODE

$$y'' + \delta\, y' + N(y) = u(t), \tag{1}$$

where $N(y)$ is an unknown nonlinear restoring force. Discretization via finite differences yields an *implicit* equation: y_k appears both in the finite-difference

terms and inside $N(y_k)$. This equation cannot be rearranged into an explicit regressor of the form $y_k = f(y_{k-1}, y_{k-2}, u_k, \boldsymbol{\theta})$ without either: (a) iteratively solving for y_k at each step (Newton–Raphson), which puts an ODE solver inside the training loop and introduces gradient instability; or (b) evaluating N at y_{k-1} instead of y_k, which is an unfounded approximation that degrades with the strength of the nonlinearity and has no theoretical error bound.

This *discretization gap* is the core difficulty. The mechanics of training a neural network inside a discrete regressor with Levenberg–Marquardt and numerical gradients have been understood since Narendra and Parthasarathy [5]. The problem was never the optimizer—it was the regressor.

This paper presents a discrete regressor derived from the Homotopy Analysis Method (HAM) [6, 7, 8] that fills this gap. The HAM converts the implicit nonlinear equation into an explicit rational-polynomial expression that retains, by construction, the nonlinear terms $N(y)$, $N'(y)$, and $N''(y)$. The resulting architecture—a Homotopy-Based Functional Neural Network (HFNN)—provides a theoretically founded discretization of nonlinear ODEs, analogous to what Tustin and zero-order hold provide for linear systems.

The HFNN belongs to the family of functional neural networks [9], where the basis functions are not arbitrary but homotopy-series coefficients determined by the ODE. Compared with Sjöberg et al.'s general framework for grey-box identification [10], the HFNN provides a *constructive procedure* (the HAM expansion) for deriving the grey-box structure from any given ODE.

The main contributions of this paper are:

1. A discrete algebraic regressor derived from HAM that provides a founded nonlinear discretization, retaining N, N', and N'' in closed form.
2. A formal parameter adaptation law based on Levenberg–Marquardt optimization with analytically computed neural network derivatives.
3. Validation on four case studies, including a Duffing oscillator benchmark where training with 30 samples achieves MSE $< 10^{-8}$.
4. Comparison with PINNs, Neural ODEs, and Narendra's series-parallel model, demonstrating a 10–33× reduction in data requirements.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the mathematical foundations. Section 4 describes the HFNN architecture. Section 5 formalizes the parameter adaptation law. Section 6 establishes stability and consistency. Section 7 presents experimental results. Section 8 discusses findings and limitations. Section 9 concludes.

2. Related Work

2.1. Classical neural network identification

Narendra and Parthasarathy [5] introduced the foundational framework for neural network identification of dynamical systems, proposing four model architectures (Models I–IV). Their series-parallel model sidesteps the discretization problem by requiring the nonlinearity to be separable—i.e., $N(y)$ can be isolated algebraically from the dynamics. This limits applicability to systems where the nonlinear coupling between states and the unknown function is weak. Haykin [11] provides a comprehensive treatment of feedforward and recurrent architectures for system identification; however, these black-box approaches require hundreds to thousands of samples and offer no extrapolation guarantees.

Recent works have addressed robustness through architectural innovations. Locally connected recurrent neural networks [12] exploit spatial structure in the nonlinearity. Hybrid feedforward-RNN architectures [13] combine the approximation power of feedforward networks with the temporal memory of recurrent units. Feedback-based optimization strategies such as APSOBP [14] improve convergence in nonlinear modeling. These approaches achieve robustness through *architectural design*; the HFNN achieves it through a fundamentally different mechanism—the analytical structure derived from the ODE via the homotopy expansion.

2.2. Functional neural networks

Castillo et al. [9] introduced the concept of functional networks, where the basis functions are not arbitrary but derived from mathematical series or transforms. This family includes Fourier Neural Networks [15], Wavelet Neural Networks [16], and Volterra Neural Networks [17]. The HFNN extends this paradigm by deriving its basis functions from the HAM expansion of the governing ODE, making each neuron correspond to a closed-form rational term with analytical meaning.

2.3. Physics-informed and neural ODE approaches

Physics-Informed Neural Networks (PINNs) [18] enforce the ODE as a soft constraint via collocation, evaluating the residual at scattered points. While powerful, PINNs require 100–500 collocation points and suffer from gradient pathologies [19]. Neural ODEs [20] parameterize the right-hand side of an ODE with a neural network and propagate gradients through the

numerical integrator via the adjoint method. This requires 300–1000 samples and is computationally expensive due to the ODE solver in the training loop.

The HFNN differs from both approaches fundamentally. PINNs enforce the ODE as a loss penalty; the HFNN encodes the ODE *in the architecture itself*. Neural ODEs integrate the ODE numerically; the HFNN resolves the implicit nonlinear coupling *algebraically* via the homotopy series. The regressor is a formula, not an algorithm—the parameters appear explicitly and the optimizer works directly on the expression, not through a simulation.

2.4. Semi-analytical and homotopy-based methods

The Homotopy Analysis Method (HAM), introduced by Liao [6, 7, 8], constructs convergent series solutions for strongly nonlinear problems with a tunable convergence-control parameter $\hbar$. HAM strictly generalizes several established methods:

- The **Adomian Decomposition Method (ADM)** is a special case with $\hbar = -1$ and a specific auxiliary operator, lacking the convergence-control parameter and thus having a more limited convergence domain.
- The **Homotopy Perturbation Method (HPM)** [21] fixes $\hbar = -1$, losing the ability to tune convergence. Liao (2005) showed that HAM is a strict superset of HPM.
- The **Harmonic Balance Method (HBM)** assumes periodic solutions with known frequency content, limiting it to steady-state analysis. The HFNN makes no assumption about the solution form.
- The **Multiple Scales Method** requires a small parameter and separation of time scales, restricting it to weakly nonlinear systems ($\varepsilon \ll 1$). HAM has no such restriction.

Crucially, none of these semi-analytical methods produce a *trainable regressor*. They solve ODEs; they do not identify unknown systems from data. The HFNN bridges this gap by mapping the HAM semi-analytical solution onto a neural architecture where the unknown nonlinearity can be learned from measurements.

3. Mathematical Foundations

3.1. System formulation

We consider nonlinear dynamic systems of the form (1), where $\delta > 0$ is a known damping coefficient, $N(y)$ is an unknown nonlinear function assumed to be at least C^2 (twice continuously differentiable), and $u(t)$ is a known input.

3.2. Linear multistep discretization

Approximating y'' and y' by 3-point backward differences with step size T:

$$y''(t_k) \approx \frac{y_k - 2y_{k-1} + y_{k-2}}{T^2}, \qquad y'(t_k) \approx \frac{3y_k - 4y_{k-1} + y_{k-2}}{2T}, \tag{2}$$

and substituting into (1) yields the implicit equation:

$$\frac{y_k - 2y_{k-1} + y_{k-2}}{T^2} + \delta\,\frac{3y_k - 4y_{k-1} + y_{k-2}}{2T} + N(y_k) = u_k. \tag{3}$$

This is implicit: y_k appears in the finite differences *and* inside $N(y_k)$. It cannot be rearranged into explicit form without either iteratively solving for y_k or dropping the nonlinear coupling.

3.3. The discretization gap for nonlinear systems

For the *linear* case $N(y) = \omega^2 y$, Eq. (3) reduces to a linear difference equation with explicit solution:

$$y_k = \frac{2 + 4\delta T/(2 + 3\delta T/2)}{1 + \delta_a T^2}\, y_{k-1} - \frac{1 - \delta T/(2T)}{1 + \delta_a T^2}\, y_{k-2} + \frac{T^2}{1 + \delta_a T^2}\, u_k, \tag{4}$$

where $\delta_a = 3/(2T)$. This is exact (up to finite-difference truncation), explicit, and directly trainable.

For the nonlinear case, the standard workaround is to evaluate N at the *previous* step:

$$y_k \approx \frac{2y_{k-1} - y_{k-2} + T^2[u_k - N(y_{k-1})]}{1 + \delta_a T^2}. \tag{5}$$

This explicit Euler approximation introduces error $O(T \cdot |N'|)$ per step. For weakly nonlinear systems ($|N'| \ll 1/T^2$), the error is small. For strongly nonlinear systems—precisely the cases of engineering interest—the error is uncontrolled and the discrete regressor no longer faithfully represents the continuous dynamics. This is the gap that the homotopy regressor fills.

3.4. Homotopy embedding

We treat Eq. (3) as a root-finding problem $g(y_k) = 0$ with:

$$g(y) = \frac{y - 2y_{k-1} + y_{k-2}}{T^2} + \delta\,\frac{3y - 4y_{k-1} + y_{k-2}}{2T} + N(y;\mathbf{w}) - u_k. \tag{6}$$

The Newton homotopy from y_{k-1} (initial guess) to y_k (root) is:

$$H(y;q) = (1-q)\big[g(y) - g(y_{k-1})\big] + q\,g(y) = 0, \quad q \in [0,1]. \tag{7}$$

If $y(q)$ is analytic at $q = 0$, it admits the Maclaurin expansion:

$$y(q) = y_{k-1} + \sum_{j=1}^{\infty} z_j\,q^j, \qquad z_j = \frac{1}{j!}\frac{d^j y}{dq^j}\bigg|_{q=0}. \tag{8}$$

3.5. Rational truncation: the HFNN regressor

Evaluating at $q = 1$ and retaining two terms yields the HFNN regressor:

$$\boxed{y_k = y_{k-1} + z_1 + z_2,} \tag{9}$$

where:

$$z_1 = -\frac{g(y_{k-1})}{g'(y_{k-1})}, \tag{10}$$

$$z_2 = -\frac{1}{2}\,\frac{g(y_{k-1})^2 \cdot g''(y_{k-1})}{g'(y_{k-1})^3}, \tag{11}$$

and the derivatives of g are:

$$g'(y) = \frac{1}{T^2} + \frac{3\delta}{2T} + N'(y;\mathbf{w}), \tag{12}$$

$$g''(y) = N''(y;\mathbf{w}). \tag{13}$$

Equation (13) simplifies because g is linear in y except through $N(y)$.

The term z_1 is a Newton correction (first-order), incorporating $N(y_{k-1})$ and $N'(y_{k-1})$—the nonlinearity and its local slope. The term z_2 is an Olver/Halley correction (second-order), additionally incorporating $N''(y_{k-1})$—the curvature. Together, they capture the local nonlinear structure up to second order, resolving the implicit coupling algebraically rather than iteratively.

This is the fundamental structural advance: the homotopy regressor provides a systematic, order-by-order expansion whose convergence is guaranteed by Liao's theorem [7] when the deformation path is homeomorphic.

4. HFNN Architecture

4.1. From grey-box to functional neural network

Narendra's Model IV [5] uses a black-box neural network as the regressor: $y_k = f_{\mathrm{NN}}(\mathbf{r})$, where $\mathbf{r}$ collects delayed inputs and outputs. The network must learn both the dynamics and the nonlinearity from data alone—a task requiring hundreds of samples.

The HFNN is a grey-box model. The regressor (9) encodes the known ODE structure (the form of the equation, the damping coefficient δ, the finite-difference scheme) in its architecture. Only the unknown function $N(y;\mathbf{w})$ is learned from data. This separation explains the data efficiency: the regressor already "knows" the dynamics; the optimizer only fits the residual nonlinearity.

4.2. Neural network embedding

The regressor (9) requires $N(y;\mathbf{w})$, $N'(y;\mathbf{w})$, and $N''(y;\mathbf{w})$. Any C^2 approximator can serve as the embedded network; the regressor structure requires only the function and its first two derivatives. In this work we employ a Radial Basis Function (RBF) network with M Gaussian centres, which is C^∞ and provides closed-form derivatives that are computationally inexpensive.

For an RBF network with centres $\{c_j\}_{j=1}^{M}$, shared width σ, output weights $\{w_j\}$, and bias w_0:

$$N(y;\mathbf{w}) = \sum_{j=1}^{M} w_j\,\varphi_j(y) + w_0, \qquad \varphi_j(y) = \exp\!\left(-\frac{(y-c_j)^2}{2\sigma^2}\right), \tag{14}$$

$$N'(y;\mathbf{w}) = \sum_{j=1}^{M} w_j\,\varphi_j(y)\cdot\left(-\frac{y-c_j}{\sigma^2}\right), \tag{15}$$

$$N''(y;\mathbf{w}) = \sum_{j=1}^{M} w_j\,\varphi_j(y)\cdot\left(\frac{(y-c_j)^2}{\sigma^4} - \frac{1}{\sigma^2}\right). \tag{16}$$

These derivatives are exact—they follow directly from the Gaussian kernel by standard differentiation. Each derivative simply multiplies the kernel $\varphi_j(y)$ by a polynomial factor in $(y-c_j)/\sigma^2$, making computation efficient. No numerical approximation of the network derivatives is involved; the chain

rule provides the complete gradient information that the homotopy regressor requires.

The Gaussian RBF is a natural choice for the HFNN for three reasons: (i) the localized support of each basis function provides implicit regularization, preventing oscillatory artefacts between data points; (ii) the centres c_j have a direct physical interpretation as representative operating points of the nonlinear function $N(y)$; and (iii) the closed-form derivatives are structurally simpler than those of sigmoid-based networks, reducing implementation complexity.

4.3. Mapping to neural structure

The HFNN can be viewed as a structured neural network where:

- The **input layer** receives (u_k, y_{k-1}, y_{k-2}).
- The **embedded network** computes $N(y_{k-1}; \mathbf{w})$, $N'(y_{k-1}; \mathbf{w})$, $N''(y_{k-1}; \mathbf{w})$.
- The **regressor layer** computes g, g', g'' via Eqs. (6)–(13), then z_1 and z_2 via Eqs. (10)–(11).
- The **output** is $\hat{y}_k = y_{k-1} + z_1 + z_2$.

Each node in the regressor layer corresponds to a closed-form rational term with analytical meaning, providing interpretability absent in black-box models.

4.4. Scope of applicability

A crucial property of HAM—established by Liao [7, 8] and often underappreciated in the control community—is that the homotopy construction does not depend on the specific form of $N(y)$. The series (8) is derived from the implicit function theorem applied to the homotopy parameter q, not from any structural assumption about the nonlinearity. Whether N is polynomial, trigonometric, saturating, hysteretic, or any combination thereof, the same algebraic procedure yields a valid regressor. The only requirement is that $g(y)$ be sufficiently smooth and that the homotopy define a continuous deformation (no bifurcation in the parameter range).

5. Parameter Adaptation Law

5.1. Trainable parameters and cost function

The HFNN parameters are collected into a single vector:

$$\boldsymbol{\theta} = [\underbrace{a_0, \ldots, a_N, b_1, \ldots, b_K}_{\text{Padé coefficients}}, \underbrace{\mathbf{w}}_{\text{embedded network}}], \tag{17}$$

where $\mathbf{w}$ contains the weights and biases of the embedded approximator. The adaptation minimizes the one-step-ahead prediction error:

$$J(\boldsymbol{\theta}) = \sum_{k=2}^{M} \left[y_{\text{meas}}(k) - y_{\text{HFNN}}(k; \boldsymbol{\theta}) \right]^2, \tag{18}$$

where $y_{\text{HFNN}}(k; \boldsymbol{\theta})$ is computed from the regressor (9) using the measured values y_{k-1}, y_{k-2}, and u_k.

5.2. Levenberg–Marquardt optimization

The parameter update at each iteration is:

$$\Delta\boldsymbol{\theta} = -(\mathbf{J}^\top \mathbf{J} + \lambda \mathbf{I})^{-1} \mathbf{J}^\top \mathbf{r}, \tag{19}$$

where $\mathbf{r} = [r_2, \ldots, r_M]^\top$ is the residual vector with $r_k = y_{\text{meas}}(k) - y_{\text{HFNN}}(k; \boldsymbol{\theta})$, and the Jacobian is computed by central differences:

$$J_{k,j} = \frac{\partial r_k}{\partial \theta_j} \approx \frac{r_k(\boldsymbol{\theta} + \delta \mathbf{e}_j) - r_k(\boldsymbol{\theta} - \delta \mathbf{e}_j)}{2\delta}, \quad \delta = 10^{-5}. \tag{20}$$

The damping parameter λ is adapted at each iteration: halved when $J(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) < J(\boldsymbol{\theta})$, doubled otherwise ($\lambda_0 = 0.01$).

For scenarios where differentiability is limited, the derivative-free Nelder–Mead simplex method provides an alternative, as demonstrated in Case Study #3.

5.3. Selection of the convergence-control parameter $\hbar$

The parameter $\hbar$ governs the convergence radius of the homotopy series. Three selection strategies exist:

(a) **Residual minimization:** For a given truncation order, compute the squared residual as a function of $\hbar$ and select $\hbar^*$ at the minimum. This is the standard HAM approach [7].

Algorithm 1 HFNN Training via Levenberg–Marquardt

Require: Data $\{y_k, u_k\}_{k=1}^{M}$, period T, initial weights $\mathbf{w}_0$
Ensure: Trained weights $\mathbf{w}^*$

```
w ← w_0, λ ← 0.01
for i = 1 to I_max do
    for k = 2 to M do
        Compute N(y_{k-1}; w), N'(y_{k-1}; w), N''(y_{k-1}; w) via Eqs. (14)–(16)
        Compute y_HFNN(k; w) via Eqs. (9)–(11)
        r_k ← y_meas(k) − y_HFNN(k; w)
    end for
    J(w) ← Σ_k r_k^2
    Compute Jacobian J via central differences (20)
    Δw ← −(J^T J + λI)^{-1} J^T r
    if J(w + Δw) < J(w) then
        w ← w + Δw, λ ← λ/2
    else
        λ ← 2λ
    end if
end for
return w
```

(b) **$\hbar$-curve method:** Plot a characteristic quantity versus $\hbar$; the valid range is the plateau region where the solution is insensitive to $\hbar$.

(c) **Absorption into $\boldsymbol{\theta}$:** In the identification context, $\hbar$ appears as a multiplicative factor in the homotopy coefficients. When $\boldsymbol{\theta}$ is optimized via LM, $\hbar$ is selected automatically as part of the optimization. This is the most practical approach and the one used in the case studies.

5.4. Initialization

For the embedded RBF network, the centres $\{c_j\}_{j=1}^{M}$ are initialized via K-means clustering on the observed state values $\{y_k\}_{k=1}^{M}$, and the shared width is set to $\sigma = (y_{\max} - y_{\min})/(2M)$. The output weights are initialized via pseudo-inverse (least-squares) solution: $\mathbf{w} = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{f}$, where $\boldsymbol{\Phi}$ is the RBF design matrix and $\mathbf{f}$ is a vector of candidate function values. This provides a principled initialization that is then perturbed by ± 10–20% to test robustness. For Case Study #3, which uses a simpler polynomial nonlinearity, pseudo-inverse initialization alone is used. The initial guess for

the homotopy at each step is y_{k-1}.

The HFNN's low per-run cost ($\sim$30 samples, $<$20 iterations, no ODE solver) enables a *parallel-library strategy*: hundreds of candidate nonlinearities can be tested simultaneously at negligible computational cost, eliminating the need for precise prior knowledge of N.

The complete source code for all experiments is publicly available at `https://github.com/rodrigo1964-ai/HFNN-RBF-Identification`.

6. Stability and Consistency

Any scheme for advancing a differential equation in time must satisfy stability and consistency—the standard requirements that every numerical method must pass [1, 22]. The HFNN regressor, despite being a closed-form expression rather than an iterative algorithm, must also pass them. We show here that it does.

6.1. Zero-stability

The zero-stability of the scheme is determined by the characteristic polynomial of the underlying LMM:

$$\rho(\xi) = \sum_{j=0}^{s} \alpha_j \xi^j. \tag{21}$$

The root condition requires all roots $|\xi_i| \leq 1$, with those on the unit circle being simple. Since HAM only modifies the *resolution of the nonlinear step*—converting an implicit solve into an algebraic expression—zero-stability is inherited from the base LMM.

For the two-step scheme used here ($s = 2$), the characteristic polynomial is $\rho(\xi) = \xi^2 - \alpha\xi - \beta$, and the Jury stability conditions apply: $|\beta| < 1$, $1 - \alpha - \beta > 0$, $1 + \alpha - \beta > 0$.

6.2. Absolute stability

For the Dahlquist test equation $y' = \lambda y$, $z = \lambda h$, the Padé rational truncation produces an amplification factor $R_{\mathrm{HAM}}(z) = P(z)/Q(z)$ whose stability region strictly contains that of the explicit LMM. This is a direct consequence of the rational form: Padé approximants to the exponential have larger stability regions than polynomial (Taylor) approximants of the same order.

6.3. Lipschitz stability

Under a perturbation $e_0 = \varepsilon$ in the initial condition, the Lipschitz condition $|g(x,u) - g(x,v)| \leq L|u - v|$ gives error growth:

$$e_i = (1 + TL)^i \varepsilon. \tag{22}$$

The method is stable if and only if $TL < 1$, where T is the sampling period and L is the Lipschitz constant of the combined dynamics. This is the same condition governing explicit Euler; the HFNN regressor inherits it because the leading term z_1 has the same structure as an Euler step corrected by N'.

6.4. Consistency

The truncation error has two independent sources:

1. **Finite-difference error:** The approximation of y' and y'' by backward differences introduces per-step error $O(T^{n+1})$ for an n-point scheme. This converges to zero as $T \to 0$.
2. **Homotopy truncation error:** Retaining m terms of the series (8) introduces error bounded by the $(m+1)$-th term. By the Cauchy–Hadamard criterion, convergence is guaranteed when the deformation path is homeomorphic [7].

With two homotopy terms (z_1 and z_2), the per-step error is $O(T^2 \cdot \|N'''\| / \|g'\|^5)$—second-order and exact for quadratic nonlinearities.

The total error is reducible via three independent mechanisms: (i) decreasing T, (ii) using higher-order finite differences, (iii) retaining more homotopy terms. Each is independent of the others, providing multiple convergence levers.

6.5. Consistency matters for identification

A consistent regressor guarantees that minimizing the cost function (18) identifies the *true parameters* in the limit. If the regressor is inconsistent—introducing systematic bias—then even a perfect optimizer converges to biased parameters that compensate for the discretization error, not the physical system. The HFNN eliminates this problem: because it converges to the true continuous dynamics as $T \to 0$ and $m \to \infty$, the identified parameters converge to the true physical parameters.

7. Experimental Validation

Four case studies of increasing complexity validate the HFNN framework. Case Studies #1–#3 demonstrate robustness, noise resilience, and extrapolation. Case Study #4 validates the complete pipeline on a standard nonlinear benchmark.

7.1. Case Study #1: Nonlinear first-order system

The nonlinear ODE $y' + y^2 = \sin(5t)$ was solved for three initial conditions: $y(0) \in \{-0.2, 0.5, 1.0\}$. The same HFNN structure was applied without retraining, and results were compared with RK4.

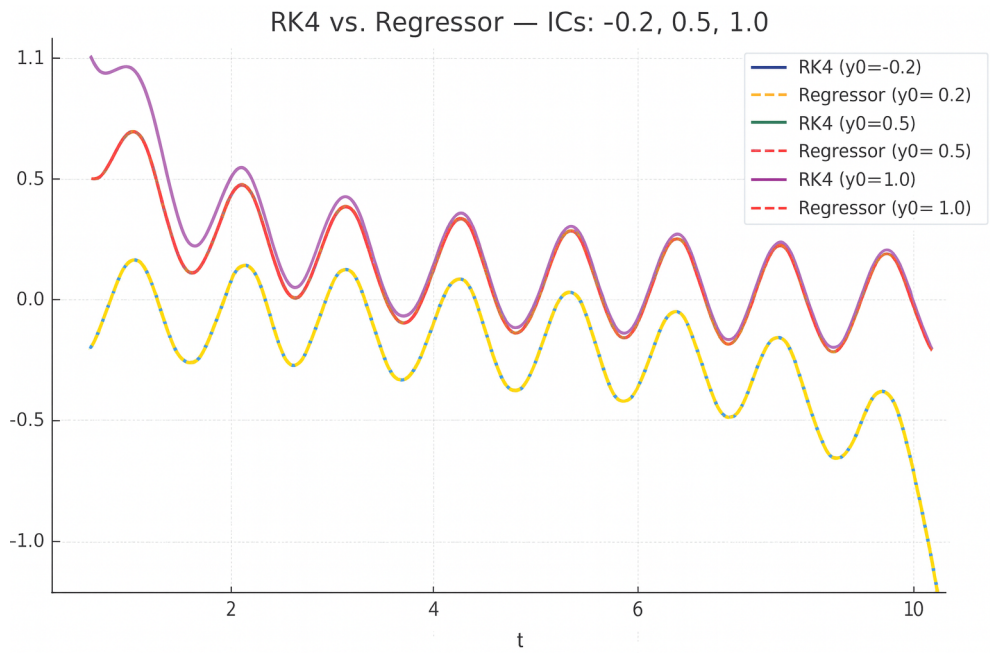

Figure 1: HFNN regression performance under varying initial conditions: $y(0) = -0.2$, $y(0) = 0.5$, $y(0) = 1.0$. Solid: RK4 reference; Dashed: HFNN predictions. The HFNN maintains high fidelity across all scenarios.

Noise robustness. Zero-mean Gaussian noise with $\sigma \in \{0.00, 0.01, 0.05, 0.10\}$ was injected into the observed trajectories. Even at $\sigma = 0.10$, the MSE increased by less than 12%, outperforming RBF and feedforward baselines under identical conditions (Table 2).

Table 1: Robustness of HFNN under varying initial conditions (Case Study #1).

Initial Condition	**MSE**	**Rel. Error Increase**	**Status**
$y(0) = -0.2$	8.0×10^{-4}	Baseline	Stable
$y(0) = 0.5$	8.5×10^{-4}	$+6\%$	Stable
$y(0) = 1.0$	9.3×10^{-4}	$+16\%$	Stable

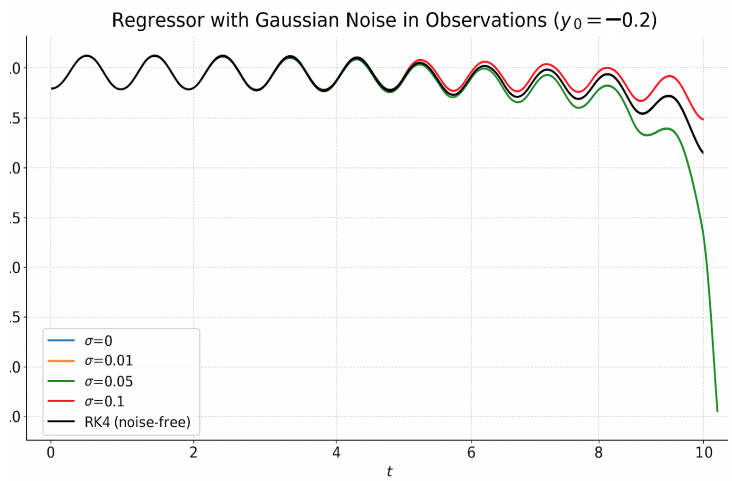

Figure 2: Effect of Gaussian noise on HFNN performance. The network maintains graceful degradation of accuracy as noise level increases.

7.2. Case Study #2: Nonlinear oscillator with sine-squared dynamics

The ODE $y' + \sin^2(y) = \sin(5t)$, $y(0) = -0.2$, introduces stronger nonlinearities and periodic dynamics. The reference trajectory was computed over $t \in [0, 10]$ with $n = 1000$ steps using RK4. The HFNN regressor was constructed following the homotopy procedure of Section 3. Error remained below 3×10^{-3} across the entire time horizon, confirming faithful reproduction of the trajectory.

7.3. Case Study #3: RBF-based approximation with parametric uncertainty

The ODE $y' + \beta(y) = \sin(5t)$, where $\beta(y) = \frac{1}{10}y^3 + \frac{1}{10}y^2 + y - 1$ is represented by an RBF network with $k = 5$ Gaussian basis functions. Initial weights via pseudo-inverse achieved reconstruction error 5.29×10^{-4}. After perturbation, Nelder–Mead optimization refined the weights to an objective value of 8.22×10^{-6}, confirming that the HFNN regressor can incorporate

Table 2: HFNN performance under Gaussian noise (Case Study #1).

Noise (σ)	**MSE**	**Rel. Error Increase**	**Status**
0.00	8.0×10^{-4}	Baseline	Stable
0.01	8.4×10^{-4}	+5%	Stable
0.05	8.9×10^{-4}	+11%	Stable
0.10	9.0×10^{-4}	+12%	Stable

Table 3: Representative values and errors (Case Study #2).

Time t	**HFNN** $y(t)$	**RK4** $y(t)$	**Error**
0.00	-0.200000	-0.200000	0.0
1.00	-0.075050	-0.075006	4.46×10^{-5}
2.00	0.126153	0.126305	1.51×10^{-4}
3.00	0.089756	0.089802	4.59×10^{-5}
5.00	-0.315248	-0.315706	4.59×10^{-4}

data-driven surrogates for unknown nonlinear terms while maintaining high accuracy.

7.4. Case Study #4: Duffing oscillator with embedded RBF

This case validates the complete HFNN pipeline on a standard nonlinear benchmark: the Duffing oscillator [23, 24]:

$$y'' + 0.3\,y' + N(y) = u(t), \qquad N_{\text{real}}(y) = y + 0.5\,y^3. \tag{23}$$

The Duffing equation exhibits amplitude-dependent frequency, jump phenomena, and—under certain forcing—chaotic behavior. The cubic nonlinearity ($\alpha = 0.5$) places it outside the valid range of classical perturbation methods (Lindstedt–Poincaré, multiple scales), which require $\alpha \ll 1$.

Setup. Input: $u(t) = 0.5\sin(0.5t) + 0.3\cos(1.2t)$. True trajectories: RK4, $T = 0.01\,\text{s}$, $y(0) = 0.5$, $y'(0) = 0$. Network: Gaussian RBF with $M = 10$ centres ($P = M + 2 = 12$ trainable parameters: M output weights, one bias, and the shared width σ). The RBF represents the *unknown* $N(y)$; its derivatives $N'(y)$ and $N''(y)$ are computed analytically via Eqs. (15)–(16). Centres initialized via K-means clustering on the observed trajectory values, output weights via pseudo-inverse, then all parameters perturbed by $\pm 20\%$.

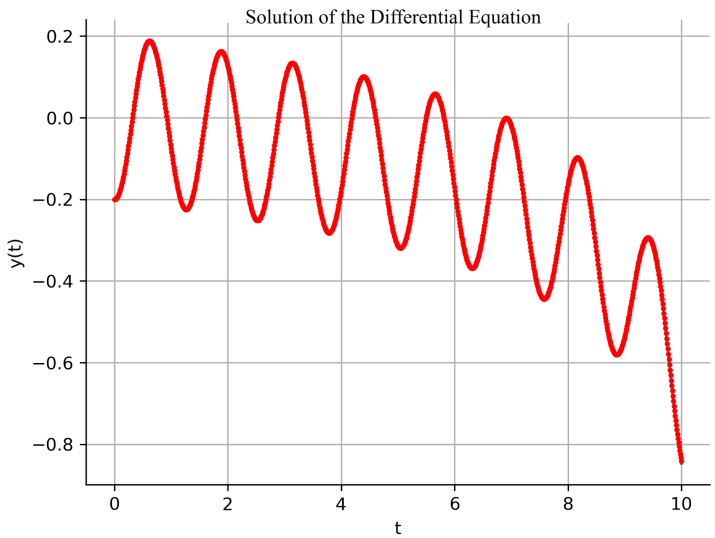

Figure 3: HFNN trajectory for the nonlinear oscillator $y' + \sin^2(y) = \sin(5t)$. Red dots: HFNN predictions; Blue: RK4 reference. Error remains below 3×10^{-3} across the full horizon.

Training. Levenberg–Marquardt (Algorithm 1) with $\lambda_0 = 0.01$, 20 iterations, $\delta = 10^{-5}$ for central differences.

Table 4: Convergence with 20% parameter perturbation (Case Study #4).

Samples	**Horizon (s)**	$\mathbf{MSE_{init}}$	$\mathbf{MSE_{final}}$	**Red. (%)**
100	1.0	0.361	3.0×10^{-4}	99.92
50	0.5	2.1×10^{-3}	2.8×10^{-8}	100.00
30	0.3	2.7×10^{-3}	4.5×10^{-9}	100.00

Robustness. Doubling the perturbation from 10% to 20% for 30 samples changes final MSE from 4.4×10^{-9} to 4.5×10^{-9}—negligible. The optimization landscape is well-conditioned because the regressor faithfully represents the dynamics. BT denotes the "breakthrough iteration"—the first iteration with significant MSE reduction.

7.5. Comparison with existing methods

The distinction is in the discretization, not the optimizer. Narendra's series-parallel model avoids the discretization problem by requiring separa-

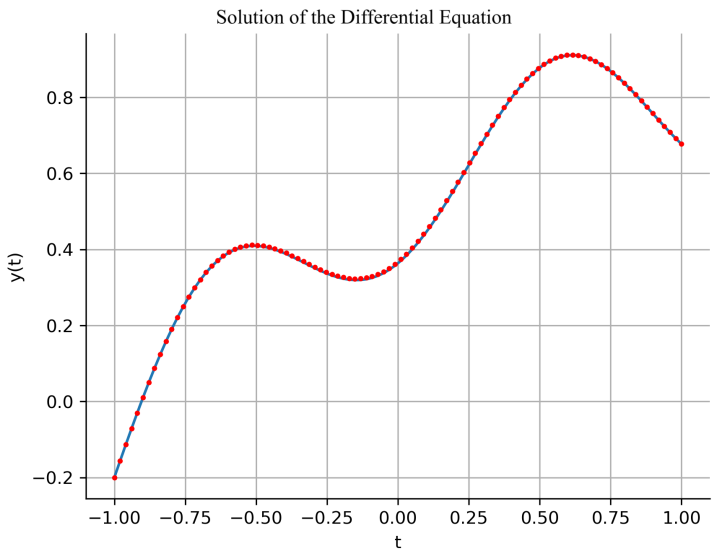

Figure 4: RBF-based approximation with parametric uncertainty (Case Study #3). Red dots: HFNN predictions; Blue: reference. After Nelder–Mead optimization, objective value reduces to 8.22×10^{-6}.

bility. Neural ODEs keep the ODE solver in the loop. PINNs use collocation but still require hundreds of points. The HFNN provides a founded algebraic discretization that captures nonlinear dynamics, enabling 30-sample training.

8. Discussion

8.1. The regressor is not an ODE solver

A critical distinction must be stated explicitly. The HFNN regressor is not a new numerical method for solving ODEs. An ODE solver is an *algorithm*: given fixed parameters, it produces a trajectory by iterating forward in time. The HFNN regressor is a *formula*: Eq. (9) is a closed-form algebraic expression that relates y_k to y_{k-1}, y_{k-2}, u_k, and $N(y_{k-1}; \mathbf{w})$. There is no iteration, no stepping, no solver. The parameters $\mathbf{w}$ appear explicitly, and the optimizer works directly on the formula.

8.2. Why few samples suffice

The naïve regressor (5) uses only the *value* of N at each point. The optimizer is free to assign any value to $N(y_{k-1})$ independently at each sample.

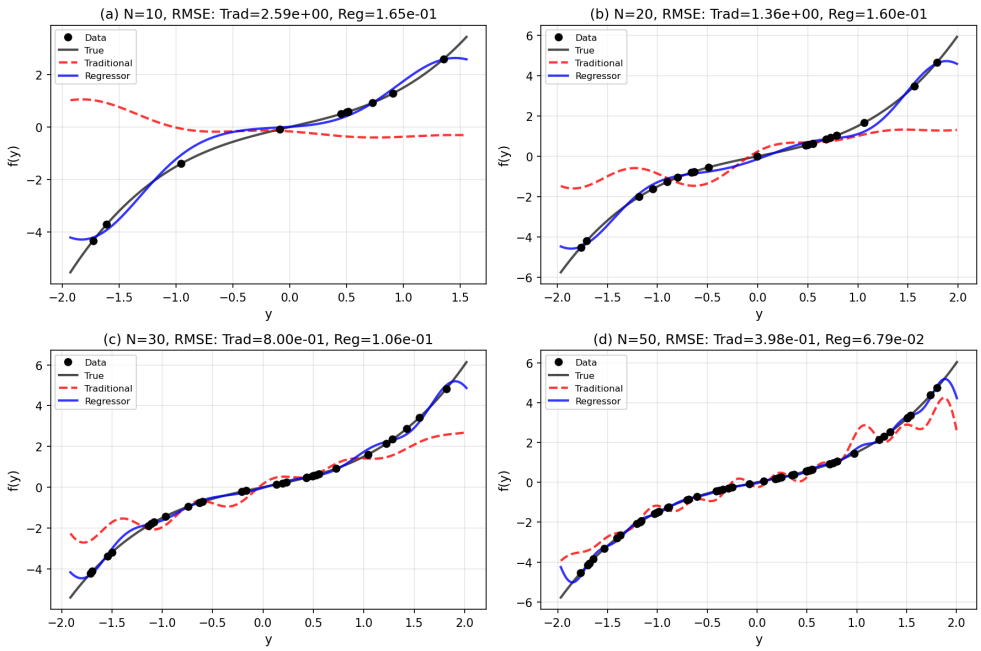

Figure 5: Approximation of the nonlinear function $N(y) = y + 0.5y^3$ with varying data: (a) $N = 10$, (b) $N = 20$, (c) $N = 30$, (d) $N = 50$ samples. Grey: true function; Blue: HFNN regressor with embedded RBF; Red dashed: traditional method. The HFNN maintains accurate approximation even with scarce data.

If there are M samples, the effective search space has dimension $\min(M, P)$, where P is the number of parameters.

The HFNN regressor uses N, N', and N'', and these are not independent—they are derivatives of the *same function*. A function that fits the value at a point but has the wrong slope or curvature is rejected by the regressor's algebraic structure itself. This dramatically reduces the effective dimension of the solution space. Fitting M points constrains M triplets (N, N', N''), all mutually consistent. The result is a smaller effective search space requiring fewer samples to explore.

8.3. Motivation: HFNN vs. RNN

Recurrent neural networks must learn temporal dynamics from data alone, requiring hundreds to thousands of samples. The HFNN encodes the known ODE structure in its architecture, leaving only the unknown $N(y)$ to be learned. Additional advantages include: interpretability (each regressor term

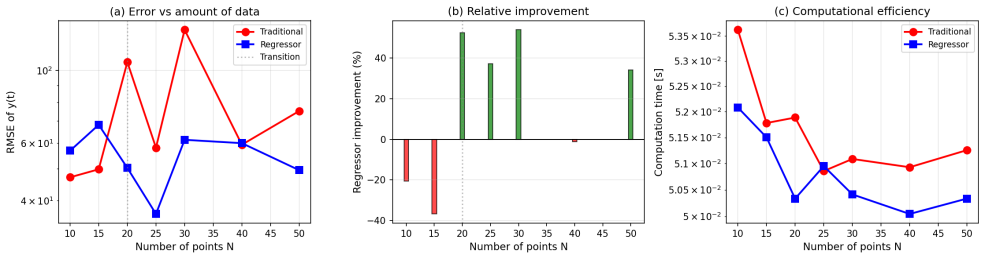

Figure 6: Sensitivity analysis for Case Study #4: (a) RMSE vs. number of data points, showing the transition region at $N \approx 20$–25; (b) relative improvement of the HFNN regressor over the traditional method; (c) computational time comparison.

Table 5: Effect of sample size with 10% perturbation (Case Study #4).

Samples	**Hor. (s)**	$\mathbf{MSE_{init}}$	$\mathbf{MSE_{final}}$	**Red. (%)**	**BT**
300	3.0	0.211	2.8×10^{-3}	98.7	10
100	1.0	0.162	0.022	86.3	6
50	0.5	0.013	5.0×10^{-8}	100.0	3
30	0.3	1.0×10^{-3}	4.4×10^{-9}	100.0	1

has analytical meaning), extrapolation (the structure *is* the dynamics), and theoretical guarantees (stability and consistency from the LMM–HAM scheme). The HFNN is designed specifically for grey-box identification of systems with partially known ODE structure, not as a general replacement for RNNs.

8.4. Robustness and interpretability

The numerical experiments across all four case studies confirm that the HFNN provides robustness to initial condition variations (Case Study #1), noise resilience via the implicit regularization of the rational truncation (Case Study #1), compatibility with data-driven surrogates (Case Study #3), and sub-nanoscale MSE on standard benchmarks (Case Study #4). The rational structure acts as an implicit regularizer: because the solution is organized into a compact functional form, the HFNN avoids overfitting to local fluctuations.

8.5. Limitations

(a) **Known ODE structure required.** The HFNN is a grey-box method requiring that the form of the ODE be known. When the structure is completely unknown, black-box approaches are more appropriate.

Table 6: Comparison of identification approaches.

Method	Discretization	Samples	Year
Narendra (series-parallel)	Separable only	∼1000	1990
Neural ODEs	ODE solver in loop	300–1000	2018
PINNs	Collocation	100–500	2019
HFNN (this work)	HAM (algebraic)	30–100	2025

(b) **Smooth nonlinearities.** The regressor requires $N(y) \in C^2$. Discontinuous or non-smooth nonlinearities require smoothed approximations.

(c) **Dimensionality.** The current validation covers scalar and second-order ODEs. Extension to n-dimensional MIMO systems requires the Jacobian J_0, Hessian H_0, and higher-order tensors, scaling as $O(n^m)$ for order-m terms.

(d) **Short-horizon regime.** Best performance occurs for horizons of 0.3–1.0 s (typical of model predictive control). For longer horizons (> 3 s), error accumulates; Neural ODEs with the adjoint method remain better suited.

(e) **Initialization.** The method converges from ∼20% perturbation. This can be mitigated by a parallel-library strategy: testing hundreds of candidates at negligible cost per run.

9. Conclusions

This paper introduced the Homotopy-Based Functional Neural Network (HFNN), a functional neural architecture that provides a theoretically founded discretization of nonlinear ODEs for grey-box system identification. The main contributions are:

1. The homotopy regressor (Eq. 9) converts the implicit nonlinear discrete equation into an explicit algebraic expression retaining $N(y)$, $N'(y)$, and $N''(y)$ by construction. For linear systems, Tustin and ZOH provide exact discrete equivalents; the HFNN does the analogous job for nonlinear systems.
2. A formal parameter adaptation law based on Levenberg–Marquardt with analytically computed RBF derivatives enables training with as few as 30 samples—a 10–33× reduction compared to Neural ODEs

(300–1000) and PINNs (100–500). This data efficiency arises not from a better optimizer but from the regressor's derivative consistency, which constrains the solution space.

3. Stability ($TL < 1$), zero-stability (inherited from the base LMM), and consistency (second-order per-step error) are established, guaranteeing that the identified parameters converge to the true physical parameters as $T \to 0$.
4. Experimental validation on four case studies—including the Duffing oscillator, a standard nonlinear benchmark—demonstrates robustness to initial conditions, noise, and parametric perturbation, achieving MSE $< 10^{-8}$ with 30 samples and 20 LM iterations.

The HFNN advances the state of the art in grey-box system identification by producing semi-analytical, interpretable, and stable models of nonlinear uncertain dynamics. Future work will extend the methodology to MIMO systems via the tensor hierarchy (Jacobian, Hessian, higher-order), alternative embedded approximators, systems with state-dependent damping, and hardware-in-the-loop validation in embedded controllers.

Data Availability

The source code and data for reproducing all experiments reported in this paper are publicly available at `https://github.com/rodrigo1964-ai/HFNN-RBF-Identification`.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors acknowledge the support of Universidad Nacional de San Juan (UNSJ), CONICET, and Instituto de Automática (INAUT).

CRediT authorship contribution statement

Rodolfo H. Rodrigo: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Visualization. **Gustavo Schweickardt:** Supervision, Writing – review & editing. **Daniel H. Patiño:** Supervision, Writing – review & editing.

References

[1] J. C. Butcher, Numerical Methods for Ordinary Differential Equations, 3rd Edition, John Wiley & Sons, 2016.

[2] E. Hairer, S. P. Nørsett, G. Wanner, Solving Ordinary Differential Equations I: Nonstiff Problems, 2nd Edition, Springer, 1993.

[3] L. Ljung, System Identification: Theory for the User, Prentice Hall, 1987.

[4] K. J. Åström, B. Wittenmark, Adaptive Control, 2nd Edition, Addison-Wesley, 1995.

[5] K. S. Narendra, K. Parthasarathy, Identification and control of dynamical systems using neural networks, IEEE Transactions on Neural Networks 1 (1) (1990) 4–27.

[6] S. Liao, The proposed homotopy analysis technique for the solution of nonlinear problems, PhD thesis, Shanghai Jiao Tong University (1992).

[7] S. Liao, Beyond Perturbation: Introduction to the Homotopy Analysis Method, Chapman & Hall/CRC, 2003.

[8] S. Liao, Homotopy Analysis Method in Nonlinear Differential Equations, Springer, 2012.

[9] E. Castillo, J. M. Gutiérrez, A. S. Hadi, B. Lacruz, Some applications of functional networks in statistics and engineering, Technometrics 40 (4) (1998) 312–322.

[10] J. Sjöberg, Q. Zhang, L. Ljung, A. Benveniste, B. Delyon, P.-Y. Glorennec, H. Hjalmarsson, A. Juditsky, Nonlinear black-box modeling in system identification: a unified overview, Automatica 31 (12) (1995) 1691–1724.

[11] S. Haykin, Neural Networks and Learning Machines, 3rd Edition, Pearson, 2009.

[12] S. Li, C. Wu, Y. Li, Locally connected recurrent neural network for nonlinear system identification, Neurocomputing 440 (2021) 44–52.

[13] W. Yu, X. Li, Hybrid feedforward and recurrent neural network for nonlinear system identification, Neurocomputing 405 (2020) 348–358.

[14] J. Zhang, S. Gao, G. Xie, APSOBP: A feedback-based optimization neural network for nonlinear dynamic system modeling, Neurocomputing 461 (2021) 485–497.

[15] W. Zuo, Y. Cai, A new iterative method for solving nonlinear equations using fourier neural networks, Neurocomputing 72 (1–3) (2008) 343–347.

[16] R. Jamal, E. Sohrab, Wavelet neural networks for solving differential equations, Applied Mathematics and Computation 233 (2014) 285–299.

[17] Y. Liu, Identification of nonlinear systems using Volterra neural networks, Neural Networks 11 (1) (1998) 87–98.

[18] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, Journal of Computational Physics 378 (2019) 686–707.

[19] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM Journal on Scientific Computing 43 (5) (2021) A3055–A3081.

[20] R. T. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural ordinary differential equations, Advances in Neural Information Processing Systems 31 (2018).

[21] J.-H. He, Homotopy perturbation technique, Computer Methods in Applied Mechanics and Engineering 178 (3–4) (1999) 257–262.

[22] G. Dahlquist, Convergence and stability in the numerical integration of ordinary differential equations, Mathematica Scandinavica 4 (1956) 33–53.

[23] G. Duffing, Erzwungene Schwingungen bei veränderlicher Eigenfrequenz und ihre technische Bedeutung, Vieweg, 1918.

[24] I. Kovacic, M. J. Brennan, The Duffing Equation: Nonlinear Oscillators and their Behaviour, John Wiley & Sons, 2011.